Smart Shipping Data Engineering Platform

From multiple sources to smarter decisions

Project Documentation: Parts 1 to 3

1. Project Planning & Management
2. Literature Review
3. Requirements Gathering

Digital Egypt Pioneers Initiative (DEPI) | Data Engineering Graduation Project | Version 1.0 | 18 October 2026

Team

#	Member	Role
1	Ziad Sameh (Team Leader)	Data Ingestion & ETL, plus coordination, repository and final documentation
2	Abdallah Gaber	Database & Data Quality
3	Ziad Reda	External Data Integration (weather)
4	Omar Mostafa	dbt & Data Modeling
5	Mahmoud Ali	Analytics & Business KPIs
6	Abdelrhman Ahmad	Airflow & Pipeline Integration

Part 1: Project Planning & Management

1.1 Project Proposal

Overview

The Smart Shipping Data Engineering Platform is an automated data pipeline that turns raw e-commerce shipping data into trusted, documented models and delivery KPIs. It collects shipping and order data from files (CSV) and enriches it with historical weather data from an API. The data is loaded raw into PostgreSQL and then transformed inside the warehouse with dbt (an ELT approach, as the lecturer advised). Data quality checks guard every stage, Apache Airflow runs the whole workflow in the right order, and the results appear in a Power BI dashboard.

Objectives

- O1. Load the CSV files into PostgreSQL with validation, logging and safe re-runs.
- O2. Design a structured PostgreSQL database with keys and relationships.
- O3. Add automatic data quality checks with clear pass or fail results.
- O4. Fetch historical weather data and join it to deliveries by date and location.
- O5. Build documented and tested dbt models (Staging, Intermediate, Marts).
- O6. Define delivery KPIs and show them in a Power BI dashboard.
- O7. Orchestrate everything with an Airflow DAG with dependencies, retries and visible errors.

Scope

In scope	Out of scope
CSV file ingestion with Python and Pandas	Real-time streaming data
Historical weather data integration from an API	Machine learning or prediction models
PostgreSQL warehouse and SQL analytics	Production cloud deployment
Data quality checks	Real or personal customer data
dbt models, tests and documentation	Payment or checkout features
Airflow DAG with retries and monitoring	Live carrier tracking
Power BI dashboard	

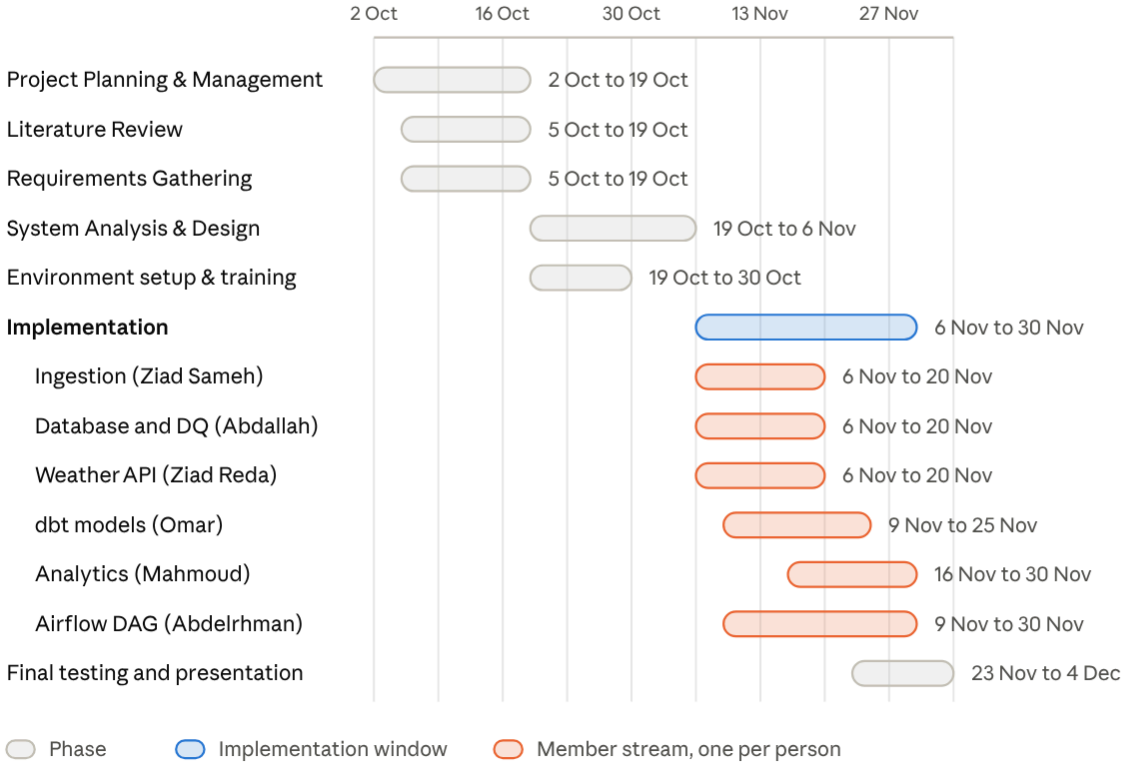
1.2 Project Plan

The plan follows the official DEPI schedule: Parts 1 to 3 first, System Analysis & Design on 6 November, Implementation on 30 November, and the final presentation, testing and reports on 4 December 2026. Because the team works with several tools at once (Airflow, dbt, PostgreSQL, Power BI), a short environment setup and training phase runs alongside System Analysis & Design.

Gantt chart

Parts 1 to 3 first, then design, a parallel build, and final testing

Project timeline, bars to scale by week



project Gantt chart · 13 bars, Oct to Dec 2026

Milestones and deliverables

Date	Milestone	Deliverable
19 Oct 2026	Documentation Parts 1 to 3 submitted	One PDF in the GitHub repository
30 Oct 2026	Environment ready	Same tool versions on all laptops; each member has run a first exercise
6 Nov 2026	System Analysis & Design complete	Use case diagram, architecture, ERD, data dictionary, wireframes
16 Nov 2026	Ingestion working	File data and weather data loaded into PostgreSQL, with logs
25 Nov 2026	Data models ready	Quality checks and dbt models with tests passing
30 Nov 2026	Implementation complete	Airflow DAG runs the pipeline end to end; dashboard built
4 Dec 2026	Final delivery	Test report, final report, presentation

Resource allocation

Resource	Allocation
People	Six team members, one stream each; the team leader also coordinates
Ingestion	Python and Pandas
Database	PostgreSQL and SQL
Weather data	Historical weather API
Transformation	dbt
Orchestration	Apache Airflow
Analytics	Power BI
Version control	Git and GitHub, one shared repository
Data	Shipping and order data from CSV files; weather data from an API; the source and licence of each are recorded in the README

1.3 Task Assignment & Roles

Member	Role	Responsibilities	Deliverables
Ziad Sameh (Team Leader)	Data Ingestion & ETL	Load the CSV files with Python and Pandas; validate files, columns and types; load into PostgreSQL; log errors and row counts; make loading re-runnable and callable from Airflow. As leader: supervise all members, track milestones and risks, own the repository, README and final documentation	Independent ingestion tasks, progress tracking, README
Abdallah Gaber	Database & Data Quality	Design PostgreSQL schemas, tables and relationships with primary and foreign keys; build checks for duplicates, missing and invalid values; define pass or fail conditions; give Airflow clear results	Organized database, automatable quality checks
Ziad Reda	External Data Integration	Call the weather API for historical weather; standardize by date and location; save raw responses; handle API errors and timeouts; make fetching re-runnable and ready for the DAG	Weather task joined with the core data

Member	Role	Responsibilities	Deliverables
Omar Mostafa	dbt & Data Modeling	Build Staging, Intermediate and Marts models; prepare fact and dimension tables; write dbt tests and documentation; prepare standalone run commands; coordinate with Abdelrhman for Airflow	Documented, testable dbt models
Mahmoud Ali	Analytics & Business KPIs	Define delay and on-time delivery indicators; analyze by region, product and seller; study weather versus delay; verify KPIs with SQL; build the Power BI dashboard; agree with Abdelrhman when data is ready	Documented KPIs, verified results, dashboard
Abdelrhman Ahmad	Airflow & Pipeline Integration	Set up Airflow; create the DAG and dependencies; connect ingestion, weather and quality tasks; run dbt models and tests after success; configure retries; monitor runs; test a forced failure; coordinate end-to-end testing	DAG that runs the project from collection to analytical models

Important: Abdelrhman is responsible for Airflow integration, but Members 1 to 4 are responsible for making their own tasks automatable and for fixing errors inside them.

1.4 Risk Assessment & Mitigation Plan

ID	Risk	Prob.	Impact	Mitigation	Owner
R1	Duplicate data when the pipeline is re-run or fails midway	Medium	High	Unique keys and upserts; every load idempotent; test re-runs	Ziad Sameh
R2	Missing or invalid values in the data	High	Medium	Document a rule per problem; quality checks with clear pass or fail; log rejected rows	Abdallah
R3	Weather API failure, timeout or request limits	Medium	Medium	Retries with timeouts; keep raw responses; reuse saved data when needed	Ziad Reda
R4	Weather data does not match delivery dates or locations	Medium	Medium	Standardize by date and location; test the join; document assumptions	Ziad Reda
R5	Stages do not fit together (names, formats)	Medium	High	One data dictionary; agreed interfaces; weekly integration test	Abdelrhman
R6	Airflow dependencies become complex	Medium	Medium	Start with a simple DAG; add tasks step by step; document dependencies	Abdelrhman
R7	Scope is too broad for the time available	High	High	Finish the core scope first; treat extra analysis as optional	Ziad Sameh
R8	Uneven workload between members	Medium	Medium	Regular progress reviews; redistribute tasks early	Ziad Sameh
R9	Team is new to Airflow, dbt or Power BI	High	High	Setup and training phase (19 to 30 Oct); one first exercise per member; use official documentation	All
R10	Passwords or API keys pushed to GitHub	Low	High	Keep secrets in local configuration excluded by .gitignore	Ziad Sameh
R11	Different tool versions on team laptops	Medium	Medium	Agree on one version set; record it in the README	Abdelrhman
R12	Time pressure from other courses	Medium	Medium	Keep tasks small; keep a buffer in the final week	All

1.5 Key Performance Indicators (KPIs)

The KPIs below measure whether the project succeeds. Targets apply to the sample data and will be checked during testing.

KPI	Target	How it is measured
Tool readiness	Every member runs a first task in their tool by 30 Oct	Short demo to the team
Pipeline success rate	At least 95% of Airflow runs finish without manual fixes	Airflow run history
Row-count accuracy	Rows loaded match rows in the source for every table	Logged row counts
Re-run safety	0 duplicate rows after repeated runs	Re-run test
Data quality pass rate	All critical checks pass; every failure has a logged reason	Quality check results
dbt test pass rate	100% of dbt tests pass	dbt test output
Failure recovery	Temporary failures recover automatically through retries	Forced-failure test
Error visibility	A forced task failure is clearly shown in the Airflow UI	Airflow UI check
KPI accuracy	100% of dashboard KPI values match SQL results	Comparison test
Dashboard response time	Each Power BI view loads in under 5 seconds	Timed test
Documentation coverage	All dbt models and main tasks documented	Review of dbt docs and README
Delivery	All deliverables submitted on the DEPI dates; all six members' work merged in one repository	Repository history

Part 2: Literature Review

2.1 Purpose and method

This review looks for projects and solutions similar to ours, lists their strengths and weaknesses, compares them with our project, and shows what we take from them. In October 2026 we reviewed public data engineering projects on GitHub and in a training course, published research on weather and delivery delays, and the Airflow and dbt documentation. The sources are listed in 2.7.

2.2 Reviewed solutions

A. Public e-commerce ELT pipeline with Airflow, dbt and PostgreSQL [1]

A containerized ELT pipeline on a public e-commerce dataset. Data lands raw first, then dbt transforms it inside a PostgreSQL warehouse through three layers, with a star schema, data-quality tests and an Airflow DAG. It also shows delivery performance by state.

Strengths: the closest to our design (ELT, dbt layers, Airflow, tests). Limitations: its description shows a single data source and no weather data, so it cannot explain why deliveries are late.

B. E-commerce pipeline that fetches data from APIs [2]

A pipeline that fetches e-commerce data from external APIs, transforms it with dbt, stores it in PostgreSQL, and serves a Streamlit dashboard. Airflow schedules it and checks data quality at each stage.

Strengths: API ingestion, validation at every stage, a working dashboard. Limitations: general e-commerce analytics, not focused on delivery performance or weather.

C. Training-course capstone: multi-source analytics platform [3]

A course project that ingests data from a transactional database, REST APIs and clickstream, transforms it through dbt layers (staging, intermediate, marts) into a cloud warehouse, and runs separate DAGs for ingestion, transformation and data quality.

Strengths: several sources and a separate quality stage. Limitations: a teaching exercise on a paid cloud warehouse, with no delivery or weather focus.

D. Weather-enriched logistics research [4]

A 2025 paper that reads logistics data from a CSV file, queries a historical weather API for each delivery location, joins the two, and trains machine learning models (logistic regression and random forest) to predict late deliveries.

Strengths: the same idea of enriching deliveries with weather by location and date. Limitations: it focuses on prediction models and uses a commercial integration platform; it does not describe quality tests or workflow orchestration.

E. Studies on weather and delivery delays [5][6][7]

A statistical study found that rain and snow, together with heavy traffic, increase parcel delivery times [5]. Another study of a supply chain found that weather affects delays even in normal conditions [6], and an industry article says that analysing historical weather helps companies anticipate disruptions [7].

Strengths: they show that the weather and delay relationship is worth testing.
Limitations: they are studies, not reusable, automated data platforms.

2.3 Comparison with our project

Criteria	A. ELT pipeline [1]	B. API pipeline [2]	D. Weather research [4]	Our platform
Automated ingestion	Yes	Yes, from APIs	Yes, CSV plus weather API	Yes, files and API
Weather data joined by date and location	No	No	Yes	Yes
Data quality checks and tests	Yes, dbt tests	Yes, at each stage	Not described	Yes, checks plus dbt tests
Orchestration with Airflow	Yes	Yes	No	Yes, with retries
Layered dbt models	Yes	Yes	No	Yes
Delivery KPIs	Partly, delivery days by state	Not the focus	Predicts late deliveries	Yes, by region, product and seller
Dashboard	Charts	Streamlit	Not a focus	Power BI
Main limitation	No weather data	Not logistics-focused	Prediction focus, commercial tooling	Prototype scale

2.4 Gap analysis and what we take from the review

- **Gap:** the pipeline projects show automation, testing and dbt layers but no weather data, while the weather research shows enrichment but not an orchestrated, tested pipeline. We found none that combines both in one open pipeline built around delivery KPIs.
- **From [1]:** the ELT approach, dbt layers, star schema, and delivery performance by region. From the Airflow [8] and dbt [9] documentation we take DAG dependencies with retries and data tests, and from [10] the star schema design.
- **From [2] and [3]:** API ingestion and a separate data-quality stage before transformation.

- **From [4], [5] and [6]:** joining weather to each delivery by location and date, and testing rain and similar variables against delays.
- **Our difference:** one automated pipeline from files and an API, through quality checks, dbt models and tests, to delivery KPIs that include the weather effect.

2.5 Suggested Improvements

Areas where the project can be enhanced after the first version:

- Add route distance and trip duration data to the delay analysis.
- Add more data sources, such as holidays or traffic, to explain delays better.
- Run dbt tests automatically on every pull request with CI/CD (GitHub Actions).
- Send alerts (email or chat) when an Airflow task fails.
- Add data freshness checks and row-count anomaly checks.
- Use incremental dbt models so transformation stays fast as data grows.
- Add Power BI pages for seller scorecards and weather impact.

2.6 Final Grading Criteria

Suggested breakdown of marks; adjust it to the official DEPI rubric.

Component	What is assessed	Weight
Documentation	Planning, requirements, roles, risks, KPIs, dbt docs, README	20%
Implementation	Ingestion, database, weather integration, dbt models, Airflow DAG, dashboard	40%
Testing	Quality checks, dbt tests, forced-failure test, end-to-end run	20%
Presentation	Demo, architecture explanation, teamwork, answers to questions	20%
Total		**100%**

2.7 References

[1] DevonSLran, "Medallion ELT pipeline on a public e-commerce dataset (Data Engineering Final Project)," GitHub. https://github.com/DevonSLran/Data_Engineering-Final_Project (accessed 18 Oct 2026).

[2] Seakty, "E-Commerce Data Engineering Pipeline (EcomBI)," GitHub. https://github.com/Seakty/EcomBI (accessed 18 Oct 2026).

[3] DataVidhya, "E-Commerce Pipeline with Airflow: E-Commerce Analytics Platform (Multi-Source, dbt, Snowflake)," course project. https://datavidhya.com/learn/projects/ecommerce-pipeline-airflow/project-ecommerce/ (accessed 18 Oct 2026).

[4] I. Zeroual and J. El Bouhdidi, "Enhancing supply chain agility with advanced weather forecasting," Int. J. Electrical and Computer Engineering, vol. 15, no. 6, pp. 5904-5913, Dec. 2025, doi: 10.11591/ijece.v15i6.pp5904-5913.

[5] A. Kosovac, A. Medic, N. Omerasevic and A. Kesko, "Statistical Modeling of Delivery Delays Due to Weather and Traffic Factors," BH Akademski Imenik, Mar. 2025. https://akademskiimenik.ba/publikacije/2cb1c63c0cd0bb4ee24738319ba0cd0ca442555d (accessed 18 Oct 2026).

[6] "Weather and supply chain performance in sport goods distribution," Int. J. Retail & Distribution Management, Feb. 2016, doi: 10.1108/IJRDM-08-2015-0113.

[7] ShipScience, "Understanding Shipment Exceptions Due to Weather Delays." https://www.shipscience.com/understanding-shipment-exceptions-due-to-weather-delays-f7440 (accessed 18 Oct 2026). Industry article.

[8] Apache Airflow, "Dags," core concepts. https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html (accessed 18 Oct 2026).

[9] dbt Labs, "Add data tests to your DAG." https://docs.getdbt.com/docs/build/data-tests (accessed 18 Oct 2026).

[10] R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Wiley, 2013.

Part 3: Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	How they use the system	What they need
Operations or logistics manager	Reads the dashboard	Clear on-time rate and delay results by region, product and seller
Business analyst	Queries marts and KPIs	Trusted, documented data; weather next to delivery data

Stakeholder	How they use the system	What they need
Data engineer (project team)	Builds and runs the pipeline	Safe re-runs, clear errors, tests and documentation
Pipeline operator	Monitors Airflow	Run status, retries and logs
Sellers and customers (indirect)	Affected by delivery decisions	Better delivery performance; protection of their data
DEPI instructors and evaluators	Assess the project	Complete documentation, a working demo, and a clear architecture

3.2 User Stories & Use Cases

A user story has the form: as a [user], I want to [action], so that [benefit].

Operations manager

ID	I want to...	So that...	Priority
US1	see the on-time delivery rate	I can judge shipping performance	Must
US2	filter delays by region	I know where to act	Must
US3	filter the dashboard by date range	I can compare periods	Should

Business analyst

ID	I want to...	So that...	Priority
US4	see delays by product category and by seller	I can find the main sources of delay	Must
US5	see weather next to delivery data	I can check if weather affects delays	Must

Data engineer

ID	I want to...	So that...	Priority
US6	re-run a load without creating duplicates	corrections are safe	Must

ID	I want to...	So that...	Priority
US7	see which quality checks failed and why	I can fix problems at the source	Must

Pipeline operator

ID	I want to...	So that...	Priority
US8	see run status and errors in Airflow	failures are noticed quickly	Must
US9	have temporary failures retried automatically	the pipeline recovers without manual work	Should

Use cases

ID	Use case	Who	Steps	Result
UC1	Load file data	Airflow, data engineer	1. Read the CSV files. 2. Validate files, columns and types. 3. Load into PostgreSQL. 4. Log errors and row counts.	Raw data in the database
UC2	Fetch weather data	Airflow, data engineer	1. Call the weather API for each date and location. 2. Save the raw response. 3. Standardize by date and location.	Weather data ready to join
UC3	Run quality checks	Airflow	1. Check duplicates, missing and invalid values. 2. Compare against pass or fail rules. 3. Report the result to Airflow.	Task marked success or failed
UC4	Build and test models	Airflow	1. Run dbt models after ingestion and checks succeed. 2. Run dbt tests.	Tested Staging, Intermediate and Marts
UC5	View KPIs	Operations manager, analyst	1. Open the Power BI dashboard. 2. Choose a region or date range. 3. Read KPIs.	Insights on delivery delays

ID	Use case	Who	Steps	Result
UC6	Handle a failure	Pipeline operator	1. Open the failed task in Airflow. 2. Read the error log. 3. Fix and re-run.	Pipeline recovers safely

3.3 Functional Requirements

Requirements are grouped by pipeline stage, and each group has one owner. Priority Must means needed for version 1; Should means important but not blocking.

A. Ingestion (owner: Ziad Sameh)

ID	Requirement	Priority
FR1	Load the CSV files with Python and Pandas and keep unchanged raw copies	Must
FR2	Validate files, columns and data types before loading	Must
FR3	Load the data into PostgreSQL, log errors and row counts, and make the load re-runnable	Must
FR4	Expose ingestion as functions that Airflow tasks can call	Must

B. Database and data quality (owner: Abdallah Gaber)

ID	Requirement	Priority
FR5	Design PostgreSQL schemas and tables with primary keys, foreign keys and relationships	Must
FR6	Detect duplicates, missing values and invalid values	Must
FR7	Define pass or fail conditions and give Airflow a clear result for each check	Must

C. Weather integration (owner: Ziad Reda)

ID	Requirement	Priority
FR8	Fetch historical weather from an API and save the raw responses	Must
FR9	Handle API errors and timeouts, and make fetching re-runnable	Must
FR10	Standardize weather data by date and location so it joins to deliveries	Must

D. Data modeling with dbt (owner: Omar Mostafa)

ID	Requirement	Priority
FR12	Build Staging, Intermediate and Marts models, including fact and dimension tables	Must
FR13	Add dbt tests and document the models and relationships	Must
FR14	Provide standalone commands to run models and tests	Must

E. Analytics and dashboard (owner: Mahmoud Ali)

ID	Requirement	Priority
FR15	Define delay and on-time delivery indicators	Must
FR16	Analyze results by region, product and seller	Must
FR17	Study the relationship between weather and delay	Must
FR18	Verify KPIs with SQL and set logical checks on the results	Must
FR19	Show KPIs and filters in a Power BI dashboard	Must

F. Orchestration (owner: Abdelrhman Ahmad)

ID	Requirement	Priority
FR20	Set up Airflow and create a DAG with task dependencies	Must

ID	Requirement	Priority
FR21	Connect ingestion, weather and quality check tasks to the DAG	Must
FR22	Run dbt models and tests only after ingestion and required checks succeed	Must
FR23	Configure retries for temporary failures and monitor runs in the Airflow UI	Must
FR24	Test a deliberate task failure and confirm the error is clearly visible	Must
FR25	Coordinate an end-to-end test of the full pipeline with all members	Must

3.4 Non-functional Requirements

ID	Quality	In simple words	How we check it
NFR1	Reliability	A failed run does not damage data, and a re-run creates no duplicates	Failure and re-run tests pass
NFR2	Data quality	Bad data is flagged with a reason, never silently changed	All checks logged; critical checks pass
NFR3	Performance	The full pipeline finishes in a reasonable time on the sample data; dashboard views open quickly	Dashboard view under 5 s; timed full run recorded
NFR4	Observability	Every task writes logs, and failures are visible in Airflow	Forced-failure test
NFR5	Maintainability	Each task runs on its own; code, SQL, dbt models and a README are in Git	Each stage runs and is documented alone
NFR6	Security	Passwords and API keys are not stored in GitHub; only public data is used	Repository review finds no secrets
NFR7	Usability	A non-technical user can read the dashboard without help	Short usability test

ID	Quality	In simple words	How we check it
NFR8	Portability	A clean setup works from the README on another laptop	Setup test on a second machine